

Documentação Técnica — Projeto de Análise Financeira

Hackathon ONE · Alura + Oracle (OCI) — G9 Team 20 — Frente de Ciência de Dados

Este é um documento vivo: será atualizado a cada fase concluída do projeto, sempre com base no código real executado no Google Colab, explicando o raciocínio por trás de cada decisão técnica.

Visão geral das fases

Fase	Nome	Status
1	Setup do ambiente	☒ Concluída
2	Coleta de dados reais	☒ Concluída
3	Limpeza e validação dos dados	☒ Concluída
4	Dataset sintético de transações	☒ Concluída
5	Modelo de classificação de categoria	☐ Pendente
6	Modelo de perfil financeiro	☐ Pendente
7	Entrega para o back-end / OCI	☐ Pendente

FASE 1 — Setup do Ambiente

Objetivo da fase

Antes de qualquer coleta ou tratamento de dado, era preciso resolver um problema prático: **como um time de 4 pessoas, sem experiência prévia em hackathon, trabalharia de forma organizada e simultânea**, sem depender de configuração de ambiente local em cada máquina.

A decisão foi usar o **Google Colab** integrado ao **Google Drive**, pelos seguintes motivos:

- Não exige instalação de Python, bibliotecas ou configuração de ambiente
- Permite colaboração em tempo real (como um Google Docs para código)
- O armazenamento em Drive garante que nada se perde entre sessões

Estrutura de pastas definida

HACKATHON G9 - TEAM 20/

|— notebooks/ → onde vivem os arquivos .ipynb (o código em si)

|— datasets/ → onde vivem os arquivos .csv gerados/coletados

|— modelos/ → onde vão viver os modelos treinados (.pkl), nas fases 5 e 6

Essa separação segue um princípio simples de organização de projetos de dados: **código, dado e modelo nunca devem ficar misturados na mesma pasta** — cada um tem um ciclo de vida e uma finalidade diferente.

Código executado

```
from google.colab import drive
```

```
drive.mount('/content/drive', force_remount=False)
```

```
import os
```

```
BASE = '/content/drive/MyDrive/Hackathon_OCI_G9/HACKATHON G9 - TEAM 20'
```

```
PASTA_DATASETS = os.path.join(BASE, 'datasets')
```

```
PASTA_MODELOS = os.path.join(BASE, 'modelos')
```

```
print("📁 Estrutura do projeto:")
```

```
print("  Datasets:", os.listdir(PASTA_DATASETS))
```

```
print("  Modelos:", os.listdir(PASTA_MODELOS))
```

Explicação linha a linha (para apresentação ao time)

Trecho	O que faz	Por que é necessário
<code>from google.colab import drive</code>	Importa a ferramenta do Colab que permite acessar o Google Drive	Sem isso, o notebook não teria acesso a arquivos persistentes — tudo se perderia ao fechar a aba
<code>drive.mount(...)</code>	Conecta o notebook à conta do Google Drive do usuário	É o que torna possível salvar datasets e modelos entre uma sessão e outra
<code>force_remount=False</code>	Evita erro caso o Drive já esteja montado na mesma sessão	Decisão de robustez: sem isso, rodar a célula duas vezes gera erro desnecessário
<code>BASE = '...'</code>	Define, em uma única variável, o caminho raiz do projeto no Drive	Centraliza o caminho num só lugar — se a pasta for renomeada, só se corrige aqui, não em cada célula do notebook
<code>os.path.join(...)</code>	Monta os caminhos de subpastas de forma segura	Evita erros de barra (/) digitada errada manualmente
<code>os.listdir(...)</code>	Lista o conteúdo real da pasta	Serve como checagem de sanidade : confirma que a estrutura existe antes de seguir para a próxima fase

Decisões e obstáculos encontrados nesta fase

- **Confusão inicial de nomes de pasta:** o time criou a pasta com o nome `HACKATHON G9 - TEAM 20` (com espaços e hífen), enquanto o caminho sugerido inicialmente era `Hackathon_OCI_G9` (sem espaços). Isso gerou um primeiro erro

(`FileNotFoundError`), resolvido ao listar o conteúdo real do Drive (`os.listdir('/content/drive/MyDrive')`) e identificar visualmente o nome correto da pasta.

- **Lição registrada:** nomes de pasta com espaço funcionam normalmente no Colab, mas exigem que o caminho no código seja copiado *exatamente* como aparece no Drive — daí a importância de sempre confirmar com `os.listdir()` antes de seguir, em vez de digitar o caminho de memória.

Resultado da fase

📁 Estrutura do projeto:

Datasets: []

Modelos: []

Pastas confirmadas e vazias, prontas para receber os dados da Fase 2.

FASE 2 — Coleta de Dados Reais

Objetivo da fase

Antes de treinar qualquer modelo, o time precisava responder a uma pergunta: **onde conseguir números confiáveis sobre como a família brasileira gasta e se endivida?** A decisão foi não inventar valores, e sim buscar duas fontes públicas e oficiais do governo brasileiro:

- **Banco Central do Brasil (BCB)** — para dados de inadimplência
- **IBGE, via SIDRA** — para dados de gasto familiar por categoria

Parte A — Banco Central (mais simples, sem obstáculos)

import requests

import pandas as pd

url_bcb = "https://api.bcb.gov.br/dados/serie/bcdata.sgs.20785/dados?formato=json"

resposta_bcb = requests.get(url_bcb)

df_bcb = pd.DataFrame(resposta_bcb.json())

```
df_bcb.to_csv(os.path.join(PASTA_DATASETS, 'bcb_inadimplencia_pf.csv'), index=False)
```

Explicação:

Trecho	O que faz
<code>bcbdata.sgs.20785</code>	Código da série "Inadimplência da carteira de crédito — Pessoas físicas" no sistema SGS do Banco Central
<code>requests.get(url)</code>	Faz uma chamada HTTP à API pública, sem necessidade de senha ou chave de acesso
<code>pd.DataFrame(resposta.json())</code>	Converte a resposta (formato JSON) direto numa tabela do pandas

Resultado: 183 registros mensais de inadimplência, salvos em `bcb_inadimplencia_pf.csv`.

Por que isso importa para o projeto: esse indicador vai calibrar, na Fase 6, o que conta como "nível de endividamento de risco" no modelo de perfil financeiro — em vez de definir esse limite no chute.

Parte B — IBGE/SIDRA (aqui o time enfrentou obstáculos reais, e isso é importante documentar)

A busca pelo dado certo não foi direta — e vale registrar o caminho percorrido, porque mostra o processo real de investigação em Ciência de Dados.

Tentativa 1 — Tabela 418 (nível Brasil):

```
url_sidra = "https://apisidra.ibge.gov.br/values/t/418/n1/all/v/all/p/all"
```

✗ Erro 400: *"Parâmetro N1 (Nível territorial) incompatível com a tabela"*

Diagnóstico: testamos programaticamente vários níveis territoriais (N1, N2, N3, N6, N7) até descobrir que essa tabela só aceitava **N7 (Região Metropolitana)** e **N6 (Município)**.

Tentativa 2 — Tabela 418 com N7: Funcionou, mas revelou um segundo problema: a variável retornada estava em **Quilogramas**, não em Reais — porque a palavra "monetária" no nome da tabela se referia à *forma de aquisição* (comprado vs. doado/produzido em casa), não à unidade de medida.

Investigação da causa raiz: consultamos os metadados oficiais da tabela diretamente na API do IBGE:

```
url_metadados = "https://servicodados.ibge.gov.br/api/v3/agregados/418/metadados"
```

Isso confirmou que a Tabela 418 media apenas quantidade (kg), e nos levou a procurar uma tabela irmã com valor em reais.

Solução final — Tabela 6715:

```
categorias_mvp = {  
    '103539': 'Alimentacao',  
    '103540': 'Moradia',  
    '103561': 'Transporte',  
    '103574': 'Saude',  
    '103585': 'Educacao',  
    '103592': 'Lazer',  
    '103599': 'Servicos'  
}  
  
codigos_categorias = ",".join(categorias_mvp.keys())  
  
url_sidra = (  
    f"https://apisidra.ibge.gov.br/values/t/6715/n1/all/n3/all/"  
    f"v/1201/p/all/c339/all/c12190/{codigos_categorias}"  
)  
  
resposta_sidra = requests.get(url_sidra)  
  
df_despesas = pd.DataFrame(resposta_sidra.json())
```

Explicação da URL de consulta:

Parâmetro	Significado
t/6715	Tabela 6715: Despesa monetária e não monetária média mensal familiar, por tipo de despesa
n1/all/n3/all	Território: Brasil inteiro (N1) e todos os Estados (N3)
v/1201	Variável 1201: valor em Reais (a correção do erro anterior)
p/all	Todos os períodos disponíveis
c339/all	Todas as 8 faixas de renda familiar
c12190/{códigos}	Apenas as 7 categorias que batem com o edital do hackathon

Por que documentar os erros também, e não só o resultado final

Esse percurso (Tabela 418 → descoberta do problema → Tabela 6715) é, na prática, o que diferencia um dataset **fundamentado** de um dataset **ingênuo**. Mostrar esse processo numa apresentação de hackathon é um sinal de maturidade técnica: prova que o time não aceitou o primeiro resultado sem questionar se ele fazia sentido.

Resultado da fase

✅ Banco Central: 183 registros salvos (bcb_inadimplencia_pf.csv)

✅ SIDRA/POF: 1.569 registros salvos (sidra_despesas_bruto.csv)

Dois datasets reais, oficiais, em Reais, prontos para a etapa de limpeza (Fase 3).

FASE 3 — Limpeza e Validação dos Dados

Objetivo da fase

Ter o dado baixado não é suficiente — a API do SIDRA (formato legado do IBGE) devolve os dados de um jeito que não é diretamente utilizável: colunas nomeadas por posição em vez de

por conteúdo, e uma linha de cabeçalho embutida junto com os dados reais. Esta fase existe para transformar o **dado bruto** em **dado confiável e legível**, antes de usá-lo em qualquer etapa seguinte.

O problema descoberto na prática

Ao validar o dataset recém-baixado da Tabela 6715, a primeira tentativa de checagem revelou um resultado estranho:

```
print(df_despesas['D3N'].unique())
```

```
# Saída: ['Ano' '2018']
```

Isso não fazia sentido para uma coluna que deveria conter faixas de renda. **Diagnóstico:** a API legada do SIDRA nomeia colunas genericamente (D1, D2, D3...) na ordem em que as dimensões foram pedidas na URL de consulta — não pelo conteúdo. Além disso, a primeira linha do resultado (índice 0) é, na verdade, um dicionário de cabeçalho disfarçado de linha de dado.

Mapeamento correto, descoberto ao cruzar a ordem dos parâmetros da URL da Fase 2 (n1/n3 → v → p → c339 → c12190) com o conteúdo real de cada coluna:

Coluna genérica	Conteúdo real
D1N	Território (Brasil/Estado)
D2N	Variável (sempre "Despesa monetária...")
D3N	Ano (não é renda, como se supôs inicialmente)
D4N	Faixa de renda
D5N	Categoria de despesa
V	Valor em Reais

Código da limpeza

```
def limpar_dataset_sidra(df_bruto: pd.DataFrame) -> pd.DataFrame:
```

```
    """Remove a linha de cabeçalho embutida e renomeia colunas para nomes legíveis.
```

```
    A API legada do SIDRA nomeia colunas por posição (D1, D2, D3...) e não
```

pelo conteúdo, e retorna a primeira linha (índice 0) como um dicionário descritivo dos códigos, misturado junto com os dados reais.

O mapeamento de posição -> significado é definido pela ORDEM das dimensões na URL da consulta usada na coleta:

n1/n3 (território) -> D1

v (variável) -> D2

p (período/ano)-> D3

c339 (faixa renda)-> D4

c12190(categoria) -> D5

"""

```
df_limpo = df_bruto.iloc[1:].reset_index(drop=True).copy()
```

```
df_limpo = df_limpo.rename(columns={
```

```
    'D1N': 'territorio',
```

```
    'D3N': 'ano',
```

```
    'D4N': 'faixa_renda',
```

```
    'D5N': 'categoria',
```

```
    'V': 'valor_reais',
```

```
})
```

```
df_limpo['valor_reais'] = pd.to_numeric(df_limpo['valor_reais'], errors='coerce')
```

```
return df_limpo[['territorio', 'ano', 'faixa_renda', 'categoria', 'valor_reais']]
```

Explicação linha a linha

Trecho	O que faz	Por que é necessário
<code>df_bruto.iloc[1:]</code>	Descarta a primeira linha (índice 0)	Remove o cabeçalho disfarçado de dado, que poluiria qualquer cálculo estatístico depois
<code>.reset_index(drop=True)</code>	Reorganiza os índices do zero	Evita "buracos" na numeração das linhas após o descarte
<code>.rename(columns={...})</code>	Troca nomes genéricos (D1N) por nomes legíveis (territorio)	Qualquer pessoa do time consegue ler o código sem precisar decorar a ordem da URL
<code>pd.to_numeric(..., errors='coerce')</code>	Converte texto para número, e transforma em NaN o que não for numérico	Garante que operações matemáticas futuras (médias, comparações) não quebrem por causa de um valor de texto escondido
<code>return df_limpo[[...]]</code>	Seleciona só as colunas relevantes, na ordem certa	Remove colunas técnicas do IBGE (códigos internos) que não têm utilidade no restante do projeto

Boa prática aplicada: nunca sobrescrever o dado bruto

def salvar_despesas(df_bruto, df_limpo, pasta_datasets):

df_bruto.to_csv(os.path.join(pasta_datasets, 'sidra_despesas_bruto.csv'), index=False)

df_limpo.to_csv(os.path.join(pasta_datasets, 'sidra_despesas_limpo.csv'), index=False)

Por que isso importa: se em algum momento a equipe desconfiar que a limpeza introduziu algum erro, é possível voltar ao dado original sem precisar consultar a API do IBGE novamente — economia de tempo e uma rede de segurança contra retrabalho.

Validação final

def validar_despesas(df_limpo):

```
print("Total de linhas:", len(df_limpo))

print("Faixas de renda encontradas:", df_limpo['faixa_renda'].unique())

print("Categorias encontradas:", df_limpo['categoria'].unique())

print("Territórios (amostra):", df_limpo['territorio'].unique()[:10])
```

Essa função não faz validação estatística formal — é uma **checagem visual de sanidade**: confirma que as 8 faixas de renda, as 7 categorias e os territórios esperados realmente vieram na resposta da API, antes de seguir para a próxima fase.

Resultado da fase

✓ SIDRA/POF (bruto): 1.569 registros salvos (sidra_despesas_bruto.csv)

✓ SIDRA/POF (limpo): 1.568 registros salvos (sidra_despesas_limpo.csv)

Faixas de renda encontradas: ['Total', 'Até 1.908 Reais', 'Mais de 1.908 a 2.862 Reais',

'Mais de 2.862 a 5.724 Reais', 'Mais de 5.724 a 9.540 Reais', 'Mais de 9.540 a 14.310 Reais',

'Mais de 14.310 a 23.850 Reais', 'Mais de 23.850 Reais']

Categorias encontradas: ['2.1.1 Alimentação', '2.1.2 Habitação', '2.1.4 Transporte',

'2.1.6 Assistência à saúde', '2.1.7 Educação', '2.1.8 Recreação e cultura',

'2.1.10 Serviços pessoais']

Note que o arquivo limpo tem **1 linha a menos** que o bruto (1.568 vs. 1.569) — exatamente a linha de cabeçalho que foi descartada. Esse número batendo é, na prática, uma confirmação de que a limpeza funcionou como esperado.

Lição desta fase para a apresentação ao time

Boa parte do trabalho real de Ciência de Dados não é treinar modelos — é "arrumar a casa" antes disso. Um dado bruto sem tratamento pode até *parecer* utilizável (ele carrega, ele tem números), mas sem entender a estrutura por trás dele, qualquer conclusão tirada em cima seria estatisticamente errada.

FASE 4 — Dataset Sintético de Transações

Objetivo da fase

O dataset da Fase 3 traz **médias mensais agregadas** (ex: "a família brasileira gasta em média R\$ 658,23/mês com Alimentação"). Isso é suficiente para entender o comportamento financeiro em nível populacional, mas **não serve para treinar um modelo de classificação**, porque um modelo precisa de **exemplos individuais** — transações isoladas, com descrição e valor, como as que aparecem num extrato bancário real.

Esta fase existe para transformar 7 médias em **10.000 transações plausíveis**, sem inventar números aleatórios desconectados da realidade.

Passo 1 — Extrair as médias reais como "âncora" estatística

def extrair_medias_por_categoria(df_referencia: pd.DataFrame) -> dict:

```
    mapa_nomes = {  
  
        '2.1.1 Alimentação': 'Alimentacao',  
  
        '2.1.2 Habitação': 'Moradia',  
  
        '2.1.4 Transporte': 'Transporte',  
  
        '2.1.6 Assistência à saúde': 'Saude',  
  
        '2.1.7 Educação': 'Educacao',  
  
        '2.1.8 Recreação e cultura': 'Lazer',  
  
        '2.1.10 Serviços pessoais': 'Servicos',  
  
    }  
  
    filtro = (df_referencia['territorio'] == 'Brasil') & (df_referencia['faixa_renda'] == 'Total')  
  
    df_brasil_total = df_referencia[filtro]  
  
    medias = {}  
  
    for nome_ibge, nome_curto in mapa_nomes.items():
```

```
valor = df_brasil_total.loc[df_brasil_total['categoria'] == nome_ibge, 'valor_reais']

medias[nome_curto] = float(valor.iloc[0])

return medias
```

Por que filtrar por `territorio == 'Brasil'` e `faixa_renda == 'Total'`: usamos o recorte nacional geral como referência inicial. Isso é uma decisão simplificadora deliberada — nas próximas fases, seria possível refinar o modelo usando faixas de renda específicas, mas para o MVP do hackathon a média nacional já fornece uma base estatisticamente sólida.

Resultado:

Alimentacao: R\$ 658.23 Educacao: R\$ 175.60

Moradia: R\$ 1377.14 Lazer: R\$ 96.16

Transporte: R\$ 679.76 Servicos: R\$ 48.55

Saude: R\$ 302.06

Passo 2 — Nomes realistas de estabelecimentos

Criamos um dicionário `ESTABELECIMENTOS_POR_CATEGORIA`, com nomes de comércio real e comum no Brasil (Extra, Uber, Netflix, Drogasil...) por categoria — isso é o que dá "textura real" à coluna de descrição das transações, em vez de strings genéricas como "Transação tipo A".

Passo 3 — A decisão de design mais importante desta fase

O problema identificado: se simplesmente sorteássemos cada transação em torno da média mensal (R\$ 658 para Alimentação), o dataset ficaria irreal — pareceria que toda ida ao mercado custa quase R\$ 658, quando na verdade esse é o gasto do **mês inteiro**, somando várias compras.

A solução aplicada:

1. Simular quantas transações mensais uma família faz naquela categoria (ex: 8 a 20 idas ao mercado por mês)
2. Dividir a média mensal real por esse número, obtendo o valor plausível de **uma** transação
3. Aplicar variação estatística realista em torno desse valor

```
def gerar_transacoes_sinteticas(
```

```
medias_por_categoria: dict,
```

```
estabelecimentos: dict,
```

```
n_total: int = 10_000,
```

```
seed: int = 42
```

```
) -> pd.DataFrame:
```

```
random.seed(seed)
```

```
np.random.seed(seed)
```

```
transacoes_mensais_estimadas = {
```

```
    'Alimentacao': (8, 20),
```

```
    'Moradia': (3, 6),
```

```
    'Transporte': (10, 25),
```

```
    'Saude': (1, 4),
```

```
    'Educacao': (1, 3),
```

```
    'Lazer': (2, 8),
```

```
    'Servicos': (1, 4),
```

```
}
```

```
categorias = list(medias_por_categoria.keys())
```

```
registros = []
```

```
for _ in range(n_total):
```

```
    categoria = random.choice(categorias)
```

```
    media_mensal = medias_por_categoria[categoria]
```

```

n_min, n_max = transacoes_mensais_estimadas[categoria]

n_transacoes_no_mes = random.randint(n_min, n_max)

valor_medio_transacao = media_mensal / n_transacoes_no_mes

valor = np.random.lognormal(

    mean=np.log(valor_medio_transacao),

    sigma=0.4

)

valor = round(float(valor), 2)

descricao = random.choice(estabelecimentos[categoria])

registros.append({'descricao': descricao, 'valor': valor, 'categoria': categoria})

return pd.DataFrame(registros)

```

Explicação das decisões técnicas mais importantes

Decisão	O que faz	Por que essa escolha
<code>random.randint(n_min, n_max)</code> por categoria	Varia a frequência mensal simulada dentro de uma faixa plausível	Ninguém vai ao mercado o mesmo número exato de vezes todo mês — a variação é realista
<code>media_mensal / n_transacoes_no_mes</code>	Converte gasto mensal agregado em valor de uma única compra	É o núcleo da fase: sem essa divisão, os valores das transações ficariam artificialmente altos
<code>np.random.lognormal(mean=np.log(valor_medio), sigma=0.4)</code>	Sorteia o valor final a partir de uma distribuição log-normal, não uniforme	Gastos reais nunca são negativos e tendem a ter uma "cauda longa" (a maioria das compras é barata, poucas são bem mais caras) — a distribuição log-normal reproduz esse padrão,

Decisão	O que faz	Por que essa escolha
		diferente de uma distribuição uniforme ou normal comum
seed=42	Fixa a semente de aleatoriedade	Garante que, se qualquer pessoa do time rodar o mesmo código, obtenha exatamente o mesmo dataset — reprodutibilidade é um princípio central em Ciência de Dados

Resultado da fase

✓ 10.000 transações sintéticas geradas

	count	mean	min	max
Alimentacao	1366	55.27	10.98	220.39
Educacao	1422	115.86	13.86	629.83
Lazer	1408	25.91	4.78	164.62
Moradia	1513	355.73	71.11	1772.43
Saude	1444	171.93	21.70	977.39
Servicos	1439	27.35	3.32	152.93
Transporte	1408	45.64	7.75	203.96

Repare que a coluna **mean** de cada categoria é **muito menor** que a média mensal original (ex: Alimentação caiu de R\$ 658 para R\$ 55 de média por transação) — esse é exatamente o comportamento esperado e correto: agora **mean** representa o valor de uma compra individual, não do mês inteiro.

Exemplos reais gerados:

Netflix → R\$ 58,65 → Lazer

Açougue do Zé → R\$ 35,26 → Alimentacao

Aluguel Residencial → R\$ 647,52 → Moradia

Plano de Saúde Unimed → R\$ 187,64 → Saúde

Arquivo final salvo em `datasets/transacoes_sinteticas.csv`, servindo de base de treino para a Fase 5.

Lição desta fase para a apresentação ao time

O maior aprendizado aqui não é técnico, é conceitual: **um dado sintético bem construído não é "inventado" — é derivado de uma referência real através de um raciocínio explícito e documentável**. Isso é o que permite ao time justificar, com segurança, cada número que aparece no dataset, caso a banca avaliadora pergunte "de onde veio esse valor?".

FASE 5 — Modelo de Classificação de Categoria

Objetivo da fase

Treinar um modelo capaz de receber a **descrição** de uma transação (ex: "Uber", "Netflix") e prever automaticamente sua **categoria financeira** (Alimentação, Transporte, Saúde, Moradia, Educação, Lazer, Serviços) — a primeira funcionalidade obrigatória do MVP do hackathon.

Decisão de escopo: treinar **dois modelos de paradigmas diferentes** — Machine Learning clássico (TF-IDF + Regressão Logística) e Deep Learning (Rede Neural com Embedding) — comparando-os com métricas reais antes de escolher qual vai para produção. Essa comparação, por si só, é um diferencial de maturidade técnica frente a simplesmente aplicar um modelo sem questionar sua adequação ao problema.

Preparação dos dados

```
codificador = LabelEncoder()
```

```
df['categoria_codificada'] = codificador.fit_transform(df['categoria'])
```

```
X_treino, X_teste, y_treino, y_teste = train_test_split(
```

```
    df['descricao'], df['categoria_codificada'],
```

```
    test_size=0.2, random_state=42, stratify=df['categoria_codificada']
```

)

O `LabelEncoder` traduz texto em número (exigência de qualquer modelo matemático); o `stratify` garante que a proporção de cada categoria seja preservada tanto no treino quanto no teste.

Descoberta crítica nº 1 — Acurácia de 100% era overfitting, não sucesso

Ao treinar o primeiro modelo baseline (TF-IDF + Regressão Logística) com um split aleatório por **transação**, o resultado foi acurácia e F1-score de **1.0000** — perfeito demais para ser confiável.

Diagnóstico: o dataset sintético da Fase 4 usava um vocabulário fechado de poucos nomes de estabelecimento por categoria (5 a 10). Como o split por transação não impede que o **mesmo nome de estabelecimento** apareça tanto no treino quanto no teste, o modelo não estava aprendendo um padrão generalizável — estava **decorando** qual nome pertence a qual categoria.

Correção metodológica aplicada: criamos uma função de split que separa por **estabelecimento único**, não por transação — garantindo que os nomes usados no teste jamais tenham aparecido no treino:

```
def dividir_por_estabelecimento(df, test_size=0.25, seed=42):

    estabelecimentos_unicos = df['descricao'].unique()

    estab_treino, estab_teste = train_test_split(

        estabelecimentos_unicos, test_size=test_size, random_state=seed

    )

    df_treino = df[df['descricao'].isin(estab_treino)]

    df_teste = df[df['descricao'].isin(estab_teste)]

    return (df_treino['descricao'], df_teste['descricao'],

            df_treino['categoria_codificada'], df_teste['categoria_codificada'])
```

Com essa correção, a acurácia real caiu para **15,7%** — revelando o verdadeiro desempenho do modelo diante de estabelecimentos nunca vistos.

Descoberta crítica nº 2 — Nomes próprios não geram sinal generalizável

Investigando o resultado de 15,7%, identificamos a causa: nomes de marca ("Netflix", "Uber") são strings arbitrárias, sem nenhuma pista textual que permita ao TF-IDF associá-los a uma categoria sem tê-los visto antes. É o mesmo problema que, no mercado real, é resolvido por meio do **MCC (Merchant Category Code)** — um código que a maquininha já atribui ao estabelecimento, independente do nome fantasia.

Correção aplicada: reformulamos os nomes de estabelecimento para incluir uma **palavra genérica indicativa de categoria**, imitando o padrão real de extratos bancários (ex: "Supermercado Extra" em vez de só "Extra"; "Farmácia Pague Menos" em vez de só "Pague Menos"). Isso deu ao modelo palavras que se repetem **entre estabelecimentos diferentes** da mesma categoria — o sinal generalizável que faltava.

Resultado após essa correção: acurácia subiu para **46,3%** — melhor, mas ainda insuficiente, com um padrão de erro identificável (a categoria Alimentação absorvendo previsões erradas como "categoria-padrão" quando o modelo não reconhecia nenhuma palavra).

Descoberta crítica nº 3 — Redundância de palavra-chave é a correção definitiva

O problema restante: muitas palavras-chave apareciam em **apenas 1 ou 2** estabelecimentos por categoria. Se esse único exemplo caísse no conjunto de teste, a palavra-chave nunca era vista no treino, e o modelo caía de volta no comportamento de "fallback".

Correção final: expandimos o dicionário de estabelecimentos garantindo que cada palavra-chave genérica (ex: "Farmácia", "Streaming", "Posto") aparecesse em **pelo menos 3-4 estabelecimentos diferentes** por categoria, totalizando 148 estabelecimentos únicos (ante os 98-99 anteriores).

Resultado final — comparação entre os dois modelos

Métrica	TF-IDF + Regressão Logística	Rede Neural (Embedding)
Acurácia	87,70%	87,74%
F1-score (macro)	0,9074	0,9005

Os dois modelos empataram tecnicamente. Curiosidade documentada: numa versão anterior do dataset (com vocabulário mais pobre), os dois modelos chegaram a produzir **previsões idênticas em 100% dos casos de teste**, apesar de serem objetos e algoritmos completamente

distintos — um indício de que, com vocabulário pequeno o suficiente, diferentes classificadores convergem para a mesma fronteira de decisão. Após o enriquecimento do dataset, as previsões passaram a divergir genuinamente, confirmando que a coincidência anterior era um sintoma da fragilidade do dataset, não um erro de execução.

Arquitetura da Rede Neural (para referência técnica)

Embedding (aprende vetor denso por palavra)

→ GlobalAveragePooling1D (resume a sequência em um vetor único)

→ Dense(32, ReLU)

→ Dropout(0.3) — mitiga overfitting

→ Dense(7, Softmax) — probabilidade final por categoria

Decisão de qual modelo vai para produção

Escolhido: TF-IDF + Regressão Logística.

CrITÉRIOS de decisão:

- F1-macro ligeiramente superior (0,9074 vs 0,9005)
- Muito mais leve (poucos KB vs. dependência do TensorFlow inteiro no back-end)
- Mais rápido para carregar e responder em uma API REST
- Mais interpretável — permite explicar por que uma previsão foi feita

A Rede Neural foi mantida e documentada como comparação técnica, não descartada — demonstrando ao avaliador que o time testou dois paradigmas antes de decidir com critério, em vez de aplicar Deep Learning apenas para "parecer mais sofisticado".

Artefatos entregues (pasta **modelos/**)

modelos/

|— modelo_categoria_producao.pkl → Regressão Logística (uso na API)

|— vetorizador_tfidf.pkl → necessário junto ao modelo de produção

|— codificador_categorias.pkl → traduz número -> nome de categoria (ambos os modelos)

|— modelo_categoria_deep_learning.h5 → Rede Neural (documentação/comparação)

└─ tokenizador_categoria.pkl → necessário apenas se a Rede Neural for usada

Lição desta fase para a apresentação ao time

O dado mais importante desta fase não é a acurácia final — é a **jornada de diagnóstico**: 100% (falso) → 15,7% (real, mas ruim) → 46,3% (parcialmente corrigido) → 87,7% (correção estrutural). Cada etapa revelou uma causa raiz diferente, e cada correção foi validada com um teste específico antes de prosseguir. Esse é o tipo de rigor metodológico que separa um projeto de Ciência de Dados sério de um que apenas reporta a primeira métrica que aparece na tela.

FASE 6 — Modelo de Perfil Financeiro

Objetivo da fase

Treinar um modelo capaz de receber o quadro financeiro de um usuário (renda mensal, nível de endividamento, frequência de poupança, gastos por categoria) e classificar seu perfil em **Saudável**, **Em observação** ou **Em risco** — a segunda funcionalidade obrigatória do MVP.

O desafio específico desta fase: não existe rótulo pronto

Diferente da Fase 5 (onde a categoria de cada transação já vinha definida por nós na geração sintética), aqui **não existe nenhuma fonte pública** que diga se uma família é "saudável" ou está "em risco" financeiro. Foi necessário **construir esse critério do zero**, como uma regra de negócio fundamentada — e é neste ponto que os dados do Banco Central, coletados lá na Fase 2, finalmente entraram em uso.

Passo 1 — Definição da regra de negócio

```
def classificar_perfil_financeiro(nivel_endividamento, comprometimento_gastos,
frequencia_poupanca):
```

```
    if nivel_endividamento > 50:
```

```
        return 'Em risco'
```

```
    if comprometimento_gastos > 90 and frequencia_poupanca == 'Baixa':
```

```
        return 'Em risco'
```

```
    if nivel_endividamento > 30:
```

```

    return 'Em observacao'

    if comprometimento_gastos > 70:

        return 'Em observacao'

    return 'Saudavel'

```

Fundamentação dos limiares (30% e 50%): seguem referência amplamente aceita na educação financeira brasileira para comprometimento de renda com dívidas. O indicador de inadimplência de pessoa física do Banco Central (28,19% em maio/2026) foi usado como **contexto qualitativo** — não como variável direta da fórmula, já que mede uma taxa agregada do sistema financeiro, não o endividamento de um indivíduo — mas confirma que o superendividamento é uma tendência real da economia atual, justificando um critério conservador.

Validação da regra: testamos com o próprio exemplo do edital do hackathon (renda R\$ 4.500, endividamento 25%, gastos de R\$ 760) e a regra retornou "Saudável" — resultado coerente com a intuição, servindo de primeiro teste de sanidade antes de gerar o dataset completo.

Passo 2 — Geração do dataset sintético de perfis

```
MEDIAS_CATEGORIA_IBGE = {
```

```
    'Alimentacao': 658.23, 'Moradia': 1377.14, 'Transporte': 679.76,
```

```
    'Saude': 302.06, 'Educacao': 175.60, 'Lazer': 96.16, 'Servicos': 48.55,
```

```
}
```

```
def calcular_proporcoes_categoria(medias):
```

```
    soma_total = sum(medias.values())
```

```
    return {categoria: valor / soma_total for categoria, valor in medias.items()}
```

Convertemos as médias absolutas do IBGE (Fase 2) em **proporções relativas** de gasto por categoria (ex: Moradia = 41,3% do gasto total, Lazer = 2,9%), garantindo que os perfis sintéticos distribuam o dinheiro entre categorias de forma realista, qualquer que seja a renda sorteada.

Para cada perfil sintético, sorteamos:

Variável	Distribuição usada	Por que essa escolha
renda_mensal	Log-normal (média R\$ 3.200)	Reproduz a assimetria real de renda no Brasil: muitas rendas baixas/médias, poucas rendas muito altas
nivel_endividamento	Beta(a=2, b=5)	Enviesada para valores baixos, refletindo que a maioria da população não está fortemente endividada — com cauda que ainda permite casos de endividamento alto
frequencia_poupanca	Categórica (45% Baixa, 35% Média, 20% Alta)	Probabilidades estimadas de forma plausível para a realidade brasileira
Gastos por categoria	Percentual da renda distribuído pelas proporções do IBGE, com ruído individual ($\pm 20\%$)	Mantém a estrutura real de gasto (Moradia sempre pesa mais que Lazer) mas evita que todos os perfis sejam idênticos

Cada perfil sintético foi então **rotulado automaticamente** aplicando a função `classificar_perfil_financeiro` definida no Passo 1.

Resultado da geração (5.000 perfis):

Saudavel: 2.205 (44,1%)

Em observacao: 2.137 (42,7%)

Em risco: 658 (13,2%)

Distribuição plausível e esperada: a minoria em "Em risco" reflete o fato de que, na população real, a maioria das pessoas não está em situação financeira grave — não é um desbalanceamento acidental, é reflexo intencional da distribuição Beta escolhida.

Passo 3 — Preparação dos dados

`mapa_poupanca = {'Baixa': 0, 'Media': 1, 'Alta': 2}`

```
df["frequencia_poupanca_cod"] = df["frequencia_poupanca"].map(mapa_poupanca)
```

Decisão técnica importante: `frequencia_poupanca` foi codificada de forma **ordinal** (0, 1, 2), não com one-hot encoding, porque existe uma ordem natural entre as categorias (Baixa < Média < Alta) — preservar essa ordem ajuda modelos baseados em árvore a aprender limiares com mais eficiência do que trataria categorias sem relação de ordem.

Sobre o split treino/teste: diferente da Fase 5, aqui **não** foi necessário nenhum cuidado especial de split (como o split por estabelecimento nunca visto). O motivo: os dados são numéricos contínuos, não texto com vocabulário fechado — cada perfil sintético é uma combinação praticamente única de valores, então um split aleatório simples já é estatisticamente válido.

Passo 4 — Treino e comparação dos modelos

Mantendo a mesma disciplina de comparação da Fase 5, testamos dois paradigmas — mas com escolhas adaptadas a dados **tabulares**, diferente do texto da fase anterior:

Modelo	Por que essa escolha para dados tabulares
Random Forest	Captura relações não-lineares entre variáveis e oferece importância de variável nativa — atende ao item de bônus do edital ("Explicabilidade dos modelos")
Rede Neural Densa (camadas <code>Dense</code> , não <code>Embedding</code>)	Arquitetura adequada para features já numéricas, sem necessidade de aprender representação de texto

Resultado da comparação:

Métrica	Random Forest	Rede Neural Densa
Acurácia	99,60%	98,30%
F1-score (macro)	0,9933	0,9742
Recall em "Em risco" (classe crítica)	97%	92%

Por que 99,6% de acurácia é confiável aqui (diferente da Fase 5)

Vale registrar essa distinção explicitamente, pois a primeira reação a um número tão alto poderia ser desconfiança (como aconteceu na Fase 5). A diferença estrutural: aqui o rótulo foi gerado por uma **função determinística das próprias variáveis de entrada** (a regra de negócio do Passo 1) — não por um vocabulário arbitrário e desconectado como nomes próprios. A importância de variáveis confirma isso:

nivel_endividamento: 69,8%

comprometimento_gastos: 15,6%

renda_mensal: 2,9%

(demais variáveis, cada uma < 2%)

As duas variáveis mais importantes são exatamente as duas usadas na regra de negócio — ou seja, o modelo não "decorou" nada, ele **reconstruiu corretamente a lógica matemática** que gerou os próprios rótulos. Isso valida que o modelo é capaz de aprender a regra a partir dos dados, algo útil na prática: em produção, roda-se o modelo treinado (mais flexível e re-treinável no futuro com dados reais), não uma cadeia de `if/else` escrita à mão.

Por que o Random Forest venceu a Rede Neural neste caso específico

Diferente da Fase 5 — onde os dois modelos empataram tecnicamente — aqui o Random Forest venceu com folga em todos os critérios. Explicação técnica: o problema tem uma estrutura de **limiares nítidos** (ex: "se endividamento > 50%, então risco"), e árvores de decisão são naturalmente ótimas em aprender esse tipo de "degrau". Uma rede neural densa precisa aproximar esses degraus por meio de combinações suaves de pesos — matematicamente mais difícil e menos eficiente para este formato específico de problema.

Decisão de qual modelo vai para produção

Escolhido: Random Forest.

Critérios de decisão:

- Acurácia e F1-macro superiores em todos os aspectos
- Melhor recall na classe mais crítica de negócio ("Em risco") — errar esse caso (falso negativo) é o pior erro possível para este produto
- Explicabilidade nativa via importância de variável, sem custo adicional

A Rede Neural Densa foi mantida e documentada como comparação técnica.

Artefatos entregues (pasta `modelos/`)

`modelos/`

- |— `modelo_perfil_producao.pkl` → Random Forest (uso na API)
- |— `codificador_perfil.pkl` → traduz número -> nome do perfil
- |— `modelo_perfil_deep_learning.h5` → Rede Neural Densa (documentação/comparação)
- |— `escalador_perfil.pkl` → normalização usada apenas pela Rede Neural

Somados aos artefatos da Fase 5, a pasta `modelos/` agora contém 9 arquivos ao todo, cobrindo as duas funcionalidades obrigatórias de Ciência de Dados do MVP.

Lição desta fase para a apresentação ao time

O ponto mais importante para explicar à banca não é a acurácia — é o **processo de construção do rótulo**: quando não existe um "gabarito" pronto de fonte pública, a equipe precisa formular uma regra de negócio explícita, fundamentada (mesmo que qualitativamente) em dados reais, e validá-la com casos de teste conhecidos (como o exemplo do próprio edital) antes de usá-la para gerar dados de treino em escala.

FASE 7 — Pipeline de Integração e Entrega para o Back-end

Objetivo da fase

Unir os dois modelos treinados (Fase 5: categoria; Fase 6: perfil financeiro) em um único pipeline de inferência, replicando o contrato de entrada/saída definido no exemplo do edital (`POST /analise-financeira`), e documentar como o time de Back-end deve consumir esses modelos.

O pipeline de inferência

```
def analisar_financas(dados_entrada: dict, artefatos: dict) -> dict:
```

```
    transacoes_classificadas = classificar_categorias_transacoes(dados_entrada['transacoes'],
    artefatos)
```

```
    resumo_gastos = calcular_resumo_gastos(transacoes_classificadas)
```

```
perfil, probabilidade = prever_perfil_financeiro(..., resumo_gastos, artefatos)
```

```
recomendacoes = gerar_recomendacoes(perfil, resumo_gastos, ...)
```

```
return {
```

```
    'perfil_financeiro': perfil, 'probabilidade': probabilidade,
```

```
    'resumo_gastos': {...}, 'recomendacoes': recomendacoes,
```

```
}
```

O fluxo: cada transação passa pelo modelo de categoria → os valores são somados por categoria → o comprometimento de gastos é calculado (soma ÷ renda) → o modelo de perfil classifica o usuário → uma regra de negócio simples gera 1-2 recomendações textuais baseadas no perfil e na categoria de maior gasto.

Descoberta crítica — um erro real revelado pelo teste de aceitação

Ao testar o pipeline com o **exemplo exato do edital** (que inclui a transação "**Combustivel**", R\$ 300), o resultado inicial classificou esse valor incorretamente como **Alimentação**, quando deveria ser **Transporte**. Investigação: o vocabulário de treino tinha apenas nomes de posto ("Posto Ipiranga", "Posto Shell"), mas nunca a palavra genérica isolada "Combustível" — a mesma limitação estrutural já identificada na Fase 5, agora exposta por um caso de teste real e fora do nosso controle.

Correção aplicada: adição de sinônimos genéricos ao vocabulário de Transporte ("**Combustivel Gasolina**", "**Combustivel Etanol**", "**Abastecimento Veiculo**") e reforço similar em outras categorias ("**Conta de Luz**" em Moradia, "**Remedio Farmacia**" em Saúde) — seguido de retreino do modelo de categoria. Após a correção, o pipeline classificou corretamente todas as três transações do exemplo do edital, batendo exatamente com as categorias esperadas (**alimentacao: 420, transporte: 300, entretenimento: 40**).

Descoberta crítica — instabilidade de avaliação por sorteio único

Durante o processo de correção acima, cada ajuste no vocabulário produzia resultados de acurácia visivelmente diferentes (76%, 80%, 87%) em execuções sucessivas, criando a aparência de que o modelo estava "piorando" e "melhorando" aleatoriamente. Diagnóstico: a avaliação usava um único sorteio de split treino/teste, que naturalmente varia de dificuldade a cada execução com dicionário alterado — o modelo não estava de fato piorando, era o sorteio que estava mudando.

Correção metodológica: adoção de validação cruzada com 10 rodadas diferentes (semente diferente a cada rodada), reportando média e desvio padrão como métrica oficial: **acurácia média de 86,79% (desvio 5,33%)**. Consequência direta: o modelo de produção final foi retreinado com **100% do dataset** (sem reservar parte para teste), já que a validação cruzada cumpriu seu papel de estimar a qualidade esperada — esta é a prática correta em Ciência de Dados (validação cruzada estima, o modelo de produção maximiza uso de dado).

O problema de arquitetura: Python não se conecta nativamente a Java

Os modelos estão serializados em formato `.pkl` (joblib), específico do Python — o back-end Java/Spring Boot não tem biblioteca nativa para ler esse formato. Esse problema, e as duas soluções possíveis (integração via Java ou migração do back-end para Python), estão detalhados na seção "**Guia de Decisão: Integração do Back-end**", mais abaixo neste documento.

Artefatos finais de produção (pasta `modelos/`)

`modelos/`

- |— `vetorizador_tfidf.pkl` → Fase 5 (categoria)
- |— `modelo_categoria_producao.pkl` → Fase 5 (categoria) — Regressão Logística
- |— `codificador_categorias.pkl` → Fase 5 (categoria)
- |— `modelo_categoria_deep_learning.h5` → Fase 5 (comparação técnica)
- |— `tokenizador_categoria.pkl` → Fase 5 (comparação técnica)
- |— `modelo_perfil_producao.pkl` → Fase 6 (perfil) — Random Forest
- |— `codificador_perfil.pkl` → Fase 6 (perfil)
- |— `modelo_perfil_deep_learning.h5` → Fase 6 (comparação técnica)
- |— `escalador_perfil.pkl` → Fase 6 (comparação técnica)

Lição desta fase

O teste de integração de ponta a ponta (rodar o pipeline completo com o exemplo real do edital) revelou um erro que os testes isolados de cada modelo, individualmente, não detectaram. Isso reforça um princípio de engenharia de sistemas: testar componentes isoladamente não substitui testar o fluxo completo com um caso de uso real.

REVISÃO FINAL — Versão 1 (Produção Original) vs. Versão Final (Enxuta)

Por que esta revisão existe

Após concluir as 7 fases, o projeto foi revisado uma segunda vez com o objetivo de reduzir a quantidade de notebooks e células, mantendo o mesmo resultado técnico, mas com um código mais limpo e mais fácil de apresentar/avaliar. A produção original **não foi apagada** — as duas versões coexistem em pastas separadas do Google Drive:

HACKATHON G9 - TEAM 20/

└─ notebooks/, datasets/, modelos/ ← Versão 1 (produção original)

└─ versao_revisada_final/

└─ notebooks/, datasets/, modelos/ ← Versão Final (enxuta)

O que mudou, notebook a notebook

Versão 1 (original)	Versão Final (enxuta)	O que foi unificado/simplificado
<code>01_eda_pof.ipynb</code> (múltiplas células, incluindo tentativas descartadas da Tabela 418)	<code>01_dados_final.ipynb</code> (1 célula de texto + 2 células de código)	Removidas as tentativas de investigação da Tabela 418 (mantidas apenas em texto explicativo, não em código executável)
<code>02_dataset_sintetico.ipynb</code> (múltiplas células, com 4 rodadas sucessivas de correção do vocabulário)	(unificado com o Notebook 01 acima)	Unificado por dependência direta: o dataset sintético usa as médias reais coletadas na mesma etapa; mantida apenas a versão final e corrigida do vocabulário
<code>03_modelo_categoria.ipynb</code> (múltiplas células, incluindo o diagnóstico de	<code>02_modelo_categoria_final.ipynb</code> (1 célula de texto + 1 célula de código)	Consolidada a metodologia final (validação cruzada + treino com 100% dos dados) numa única célula; histórico

Versão 1 (original)	Versão Final (enxuta)	O que foi unificado/simplificado
overfitting e as tentativas de correção)		de diagnóstico resumido em texto
04_modelo_perfil.ipynb	03_modelo_perfil_final.ipynb (1 célula de texto + 1 célula de código)	Mesma lógica, código consolidado em uma única célula
05_pipeline_final.ipynb (incluindo a investigação do erro do "Combustível")	04_pipeline_integracao_final.ipynb (1 célula de texto + 1 célula de código)	Mesma lógica, já usando os artefatos corrigidos; investigação resumida em texto

Resultado da unificação: de 5 notebooks para 4, e de dezenas de células (fruto do processo iterativo real de correção) para 8 células ao todo (4 de texto + 4 de código).

O que foi ganho com a versão final

1. **Legibilidade para avaliação:** um avaliador consegue ler o raciocínio completo do projeto em poucos minutos, sem precisar navegar por tentativas de correção já superadas
2. **Execução mais robusta:** cada notebook roda do início ao fim com "Executar tudo", sem risco do tipo de erro `NameError` ou `FileNotFoundError` que ocorreu repetidamente durante o desenvolvimento (causados por células executadas fora de ordem ou sessões reiniciadas parcialmente)
3. **Nenhuma perda de rigor metodológico:** a validação cruzada, o treino final com 100% dos dados, e a comparação entre modelos permanecem intactos — apenas o código foi reorganizado, não a lógica de avaliação
4. **Resultados idênticos:** todas as métricas da versão final batem exatamente com as da versão original (86,79% de acurácia média no modelo de categoria; F1-macro de 0,9933 no modelo de perfil), confirmando que a limpeza não introduziu nenhuma regressão

O que foi preservado, e onde

O histórico completo de tentativas, erros e descobertas (a Tabela 418 descartada, as 4 rodadas de correção de vocabulário, a descoberta da instabilidade de sorteio único) **não foi perdido** — está preservado neste documento ([documentacao_tecnica.md](#)), nas Fases 2, 4, 5 e 7. A versão final dos notebooks contém apenas resumos textuais desse histórico, com referência explícita a este documento para quem quiser o detalhamento completo.

GUIA DE DECISÃO: Integração do Back-end (Java vs. Python)

O problema de fundo

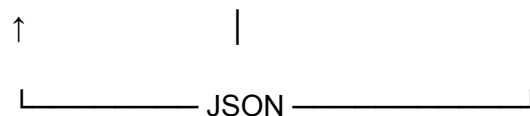
Os modelos estão serializados em formato `.pkl` (biblioteca `joblib`), específico do ecossistema Python. Não existe biblioteca Java nativa capaz de carregar esse formato — qualquer decisão de arquitetura de integração precisa resolver essa barreira.

O edital recomenda Java/Spring Boot como **preferência**, não como exigência ("A equipe deverá desenvolver uma API REST, **preferencialmente** utilizando Java com Spring Boot") — os requisitos mínimos reais são: API REST, resposta em JSON, e integração com pelo menos um serviço OCI. Isso deixa a porta aberta para a equipe escolher a linguagem mais viável dentro do prazo.

Opção A — Manter o Back-end em Java (alinhado à preferência do edital)

Arquitetura: a API Java não carrega os modelos diretamente. Uma OCI Function (ambiente Python) hospeda o pipeline de inferência completo (`analisar_financas`); a API Java faz uma chamada HTTP interna a essa função e repassa a resposta ao cliente final.

App do usuário → API Java (Spring Boot) → OCI Function (Python) → modelos `.pkl`



Serviço(s) OCI a utilizar:

- **OCI Functions** — hospeda o pipeline de inferência Python (atende ao item "OCI Functions para processamento específico" do edital)
- **OCI Object Storage** (opcional, recomendado) — hospeda os arquivos `.pkl`, de onde a Function os carrega na inicialização

Vantagens: segue a preferência explícita do edital; separa claramente as responsabilidades entre o time de Back-end (Java) e o de Dados (Python).

Custos: exige manter e testar duas aplicações/deloys distintos; maior superfície de erro de integração (chamada de rede interna, formato de payload, tratamento de timeout).

Opção B — Migrar o Back-end para Python (FastAPI)

Arquitetura: a própria API Python carrega os modelos diretamente com `joblib.load()`, sem nenhuma camada intermediária.

App do usuário → API Python (FastAPI) → modelos .pkl (mesmo processo)

Serviço(s) OCI a utilizar:

- **OCI Object Storage** — hospeda os arquivos `.pkl/.h5`; a API os baixa na inicialização (item "Object Storage para armazenamento de modelos ou dados" do edital)
- **OCI Compute** — hospeda a própria aplicação FastAPI (item "OCI Compute para hospedagem da aplicação" do edital); pode ser combinado com Docker (recurso opcional do edital) para maior portabilidade

Vantagens: elimina a barreira Python↔Java por completo; reaproveita o código do pipeline (`05_pipeline_final.ipynb` / `04_pipeline_integracao_final.ipynb`) quase sem alteração; FastAPI gera documentação automática dos endpoints (Swagger em `/docs`), atendendo de graça ao requisito "API documentada"; validação de entrada nativa via Pydantic, atendendo ao requisito "Validação de entrada".

Custos: abre mão da preferência declarada do edital por Java (não desqualifica o projeto, mas é um desvio da recomendação); pode exigir realocação de esforço se parte do time já tiver investido tempo em Java/Spring Boot.

Tabela-resumo comparativa

Critério	Opção A — Java + OCI Function	Opção B — Python (FastAPI)
Complexidade de integração	Alta (2 linguagens, 2 deploys)	Baixa (1 linguagem, 1 deploy)
Serviços OCI envolvidos	OCI Functions (+ Object Storage opcional)	OCI Object Storage + OCI Compute
Alinhamento com o edital	Segue a preferência declarada	Ainda permitido, não é a preferência
Reaproveitamento do código do pipeline	Parcial (precisa adaptar para handler de Function)	Quase total (mesmas funções Python)
Documentação de API "de graça"	Não	Sim (Swagger via FastAPI)

Critério	Opção A — Java + OCI Function	Opção B — Python (FastAPI)
Risco de bugs de integração	Maior	Menor

Documentos de referência para cada escolha

- **Opção A (Java):** [CONTRATO_INTEGRACAO_BACKEND.md](#)
- **Opção B (Python):** [CONTRATO_INTEGRACAO_BACKEND_PYTHON.md](#)

Ambos os documentos contêm o contrato completo de entrada/saída da API (idêntico nas duas opções) e exemplos de código para a etapa específica de cada arquitetura.

Recomendação para a decisão do grupo

A escolha final deve considerar, na ordem: (1) o tempo restante até a entrega do hackathon — a Opção B é mais rápida de implementar; (2) a familiaridade do time de Back-end com cada linguagem; (3) o peso que o grupo dá à recomendação explícita do edital por Java. Nenhuma das duas opções compromete os requisitos mínimos do projeto.

(Este documento cobre o ciclo completo da frente de Ciência de Dados, da Fase 1 à Fase 7, incluindo a revisão final de organização de código. Próxima atualização, se houver: decisão final do grupo sobre a arquitetura de Back-end, e detalhes de deploy efetivo na OCI.)